

# Teoria da Computação

## Reduções, P, NP, NP-difícil e NP-completo

Prof. Me. João Paulo Biazotto

Universidade Estadual do Paraná – Campus Apucarana  
Bacharelado em Ciência da Computação

Teoria da Computação  
2026

# Objetivos da aula

Ao final da aula, espera-se que vocês consigam:

- explicar o que significa reduzir um problema a outro em tempo polinomial;
- identificar a direção correta de uma redução;
- diferenciar resolver, verificar e reduzir um problema;
- explicar as classes  $P$  e  $NP$ ;
- diferenciar problema NP-difícil e problema NP-completo;
- acompanhar uma redução detalhada de Ciclo Hamiltoniano para TSP;
- entender por que TSP de decisão é NP-completo e TSP de otimização é NP-difícil.

# Roteiro

- 1 Recapitulando decidibilidade.
- 2 Reduções entre problemas.
- 3 Exemplo detalhado: Ciclo Hamiltoniano  $\leq_p$  TSP.
- 4 Classe  $P$ .
- 5 Classe  $NP$ .
- 6 NP-difícil e NP-completo.
- 7 Exemplos: TSP de decisão e TSP de otimização.

## De decidibilidade para complexidade

Na aula anterior, a pergunta central era:

Existe algum algoritmo que sempre termina e responde corretamente?

Agora, para os problemas decidíveis, fazemos uma nova pergunta:

Existe um algoritmo eficiente?

### Síntese

Computabilidade pergunta se há solução algorítmica. Complexidade pergunta quanto custa resolver o problema.

## Dois tipos de limite

| Tipo de discussão | Pergunta central                             |
|-------------------|----------------------------------------------|
| Decidibilidade    | O problema possui algoritmo que sempre para? |
| Complexidade      | Se possui algoritmo, ele é eficiente?        |

### Ideia importante

Um problema pode ser decidível, mas ainda assim ser muito caro de resolver na prática.

## Por que as reduções aparecem aqui?

Reduções são úteis em dois contextos.

### Em decidibilidade

Usamos reduções para mostrar que certos problemas são indecidíveis, transferindo a impossibilidade de um problema conhecido para outro.

### Em complexidade

Usamos reduções para comparar dificuldade entre problemas decidíveis, especialmente para definir NP-difícil e NP-completo.

### Síntese

A mesma ferramenta aparece primeiro em computabilidade e depois em complexidade.

# O que é uma redução?

## Conceito

Reduzir um problema  $A$  a um problema  $B$  significa transformar cada instância de  $A$  em uma instância de  $B$ , de modo que resolver  $B$  também resolva  $A$ .

instância de  $A \xrightarrow{f}$  instância de  $B$

A transformação deve preservar a resposta:

$$x \in A \iff f(x) \in B$$

## Redução em tempo polinomial

Dizemos que:

$$A \leq_p B$$

quando existe uma função  $f$ , computável em tempo polinomial, tal que:

$$x \in A \iff f(x) \in B$$

### Leitura

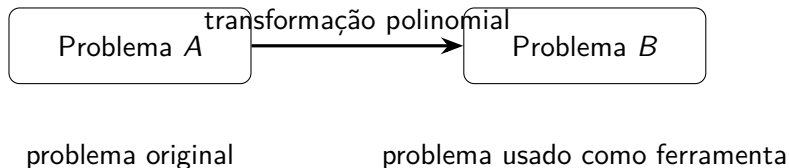
$A \leq_p B$  significa: “ $A$  reduz-se a  $B$  em tempo polinomial”.

### Interpretação

Se eu sei resolver  $B$ , posso resolver  $A$ : transformo a entrada de  $A$  em uma entrada de  $B$  e uso o algoritmo de  $B$ .



## A direção da redução



$$A \leq_p B$$

### Síntese

A seta aponta para o problema que gostaríamos de saber resolver. Se resolver  $B$  é fácil, então resolver  $A$  também fica fácil.

## O que uma redução NÃO significa

- Não significa que  $A$  e  $B$  são o mesmo problema.
- Não significa que  $A$  foi resolvido diretamente.
- Não significa que  $A$  ou  $B$  estão em  $P$ .
- Não significa que a solução de  $B$  foi construída.

### Cuidado

Quando dizemos que  $A \leq_p B$ , o que é polinomial é a transformação de instâncias, não necessariamente a resolução de  $A$  ou  $B$ .

## Como provar uma redução

Para provar que  $A \leq_p B$ , precisamos mostrar três coisas.

- ① **Construção:** como transformar uma instância  $x$  de  $A$  em uma instância  $f(x)$  de  $B$ .
- ② **Correção:** por que a transformação preserva a resposta.

$$x \in A \iff f(x) \in B$$

- ③ **Tempo:** por que  $f$  pode ser calculada em tempo polinomial.

### Síntese

Uma redução não é apenas uma intuição. Ela precisa preservar respostas SIM e NÃO.

## Primeiro problema: Ciclo Hamiltoniano

### Conceito

Um ciclo Hamiltoniano em um grafo é um ciclo que visita todos os vértices exatamente uma vez e retorna ao vértice inicial.

### Exemplo

Se o grafo possui vértices  $A, B, C, D$ , o percurso

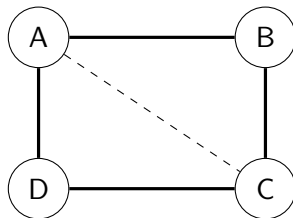
$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

é Hamiltoniano se todas essas arestas existem e nenhum vértice foi repetido antes do retorno a  $A$ .

### Problema de decisão

Dado um grafo  $G$ , existe um ciclo Hamiltoniano em  $G$ ?

## Ciclo Hamiltoniano: exemplo visual



Neste grafo, existe o ciclo:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

Ele passa por todos os vértices uma única vez e retorna ao início.

## Segundo problema: TSP

TSP significa *Travelling Salesman Problem*, ou Problema do Caixeiro Viajante.

### Versão de otimização

Dadas cidades e distâncias entre elas, encontrar a rota de menor custo que visita todas as cidades exatamente uma vez e retorna à cidade inicial.

### Versão de decisão

Dado um grafo completo ponderado e um valor  $k$ , existe uma rota que visita todos os vértices exatamente uma vez, retorna ao início e tem custo total  $\leq k$ ?

### Síntese

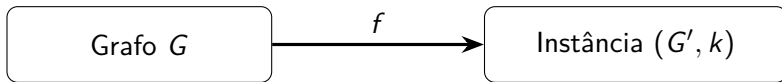
Para discutir  $P$ ,  $NP$  e NP-completude, usamos a versão de decisão.

## Objetivo da redução

Queremos provar:

$$HAM \leq_p TSP$$

Ou seja:



A transformação deve satisfazer:

$$G \text{ tem ciclo Hamiltoniano} \iff (G', k) \text{ tem rota TSP de custo} \leq k$$

# A transformação

Entrada original:

$$G = (V, E)$$

com  $n = |V|$ .

Construímos um grafo completo  $G'$  com os mesmos vértices.

Para cada par de vértices  $u, v$ :

$$c(u, v) = \begin{cases} 1, & \text{se } (u, v) \in E \\ 2, & \text{se } (u, v) \notin E \end{cases}$$

Definimos:

$$k = n$$



## Por que pesos 1 e 2?

Uma rota do TSP que visita todos os  $n$  vértices e retorna ao início usa exatamente  $n$  arestas.

- Se todas as arestas têm peso 1, o custo total é  $n$ .
- Se ao menos uma aresta tem peso 2, o custo total é maior que  $n$ .

Como definimos  $k = n$ , uma solução de custo  $\leq n$  só pode usar arestas de peso 1.

### Síntese

As arestas de peso 1 representam exatamente as arestas que existiam no grafo original.

## Exemplo concreto da transformação

Grafo original:

$$V = \{A, B, C, D\}, \quad E = \{AB, BC, CD, DA, AC\}$$

Arestas existentes recebem peso 1; arestas inexistentes recebem peso 2.

| Aresta | Existia em $G$ ? | Peso em $G'$ |
|--------|------------------|--------------|
| AB     | sim              | 1            |
| AC     | sim              | 1            |
| AD     | sim              | 1            |
| BC     | sim              | 1            |
| BD     | não              | 2            |
| CD     | sim              | 1            |

Como  $n = 4$ , temos  $k = 4$ .

## Testando o exemplo

Considere a rota:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

O custo é:

$$AB + BC + CD + DA = 1 + 1 + 1 + 1 = 4$$

Como  $k = 4$ :

$$4 \leq k$$

Logo, a instância de TSP responde SIM.

### Síntese

Essa rota corresponde exatamente a um ciclo Hamiltoniano no grafo original.

## Correção da redução: primeira direção

Queremos mostrar:

$$G \in HAM \Rightarrow f(G) \in TSP$$

Suponha que  $G$  possui um ciclo Hamiltoniano.

- Esse ciclo visita todos os  $n$  vértices.
- Portanto, usa exatamente  $n$  arestas.
- Todas essas arestas existem em  $G$ .
- Na transformação, cada uma recebeu peso 1.

Logo, o custo total é:

$$1 + 1 + \cdots + 1 = n$$

Como  $k = n$ , existe uma rota de custo  $\leq k$ .

## Correção da redução: segunda direção

Agora queremos mostrar:

$$f(G) \in TSP \Rightarrow G \in HAM$$

Suponha que  $(G', k)$  possui uma rota de custo  $\leq n$ .

- A rota visita todos os  $n$  vértices e retorna ao início.
- Portanto, usa exatamente  $n$  arestas.
- Cada aresta tem custo 1 ou 2.
- O menor custo possível para  $n$  arestas é  $n$ .

Se o custo é  $\leq n$ , todas as arestas usadas têm custo 1.

Portanto, todas existiam no grafo original  $G$ .

## Conclusão da prova de correção

Da segunda direção, concluímos que a rota do TSP:

- usa apenas arestas originais de  $G$ ;
- visita todos os vértices;
- visita cada vértice uma vez;
- retorna ao início.

Logo, essa rota é um ciclo Hamiltoniano em  $G$ .

Portanto:

$$G \in HAM \iff f(G) \in TSP$$

## Síntese

A transformação preserva respostas SIM e respostas NÃO.

## Tempo da transformação

Se o grafo possui  $n$  vértices, precisamos definir o peso de cada par de vértices.  
A quantidade de pares é:

$$\frac{n(n-1)}{2}$$

Isso é da ordem de:

$$O(n^2)$$

Portanto, a transformação pode ser feita em tempo polinomial.

$$\boxed{HAM \leq_p TSP}$$

## O que essa redução prova?

### Importante

A redução  $HAM \leq_p TSP$  não prova que  $HAM$  ou  $TSP$  podem ser resolvidos em tempo polinomial.

Ela prova que:

- transformar uma instância de  $HAM$  em uma instância de  $TSP$  é polinomial;
- se tivéssemos um algoritmo polinomial para  $TSP$ , também teríamos um algoritmo polinomial para  $HAM$ ;
- portanto,  $TSP$  é pelo menos tão difícil quanto  $HAM$ .



## Complexidade de tempo

O tempo de execução de um algoritmo pode ser expresso por uma função do tamanho da entrada.

### Exemplo

Ordenar um vetor de 5 posições tende a ser mais rápido do que ordenar um vetor de 1000 posições. O tamanho da entrada influencia o tempo necessário.

Chamamos esse tamanho de entrada de  $n$ .

Exemplos de funções de tempo:

$$n, \quad n^2, \quad n^3, \quad 2^n, \quad n!$$

# Tempo polinomial e exponencial

## Tempo polinomial

O tempo pode ser limitado por uma função polinomial, como  $n$ ,  $n^2$ ,  $n^3$ ,  $5n^2 + 8n + 4$ .

## Tempo exponencial

O tamanho da entrada aparece no expoente, como em  $2^n$  ou  $3^n$ .

## Exemplo

Para  $n = 1000$ ,  $n^3$  é grande, mas  $2^n$  é incomparavelmente maior.

# Classe P

## Conceito

A classe  $P$  contém os problemas de decisão que podem ser resolvidos por um algoritmo determinístico em tempo polinomial.

Em termos intuitivos:

$P$  = problemas decidíveis de forma eficiente

## Exemplos comuns

- verificar se existe caminho entre dois vértices de um grafo;
- encontrar menor caminho com Dijkstra, quando os pesos são não negativos;
- calcular máximo divisor comum;
- ordenar uma lista.

## Exemplo de problema em P

Problema de decisão:

Dado um grafo  $G$  e vértices  $s$  e  $t$ , existe caminho de  $s$  até  $t$ ?

Algoritmo:

- 1 execute BFS ou DFS a partir de  $s$ ;
- 2 marque todos os vértices alcançáveis;
- 3 se  $t$  foi marcado, responda SIM;
- 4 caso contrário, responda NÃO.

Como cada vértice e aresta é explorado um número limitado de vezes, o algoritmo é polinomial.

# Classe NP

## Conceito

A classe *NP* contém os problemas de decisão cujas soluções candidatas podem ser verificadas em tempo polinomial.

## Cuidado

*NP* não significa “não polinomial”. Significa que uma solução pode ser verificada em tempo polinomial por uma máquina determinística, ou resolvida em tempo polinomial por uma máquina não determinística.

## Síntese

Em aula, a ideia mais intuitiva é:  $NP =$  problemas em que é fácil conferir uma solução candidata.

## Resolver versus verificar

| Ação      | Significado                                    |
|-----------|------------------------------------------------|
| Resolver  | encontrar uma solução a partir da entrada      |
| Verificar | conferir se uma solução candidata está correta |

### Exemplo

Encontrar um ciclo Hamiltoniano pode ser difícil. Mas, se alguém fornecer uma sequência de vértices, verificar se ela é um ciclo Hamiltoniano é relativamente simples.

## Por que Ciclo Hamiltoniano está em NP?

Suponha que alguém forneça uma solução candidata:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

Para verificar, basta conferir:

- 1 todos os vértices aparecem;
- 2 nenhum vértice foi repetido, exceto o retorno ao início;
- 3 cada par consecutivo possui uma aresta no grafo;
- 4 o último vértice possui aresta para o primeiro.

Tudo isso pode ser feito em tempo polinomial.

$$HAM \in NP$$

## Por que TSP de decisão está em NP?

Entrada do TSP de decisão:

$$(G, k)$$

Solução candidata:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

Verificação:

- 1 conferir se todas as cidades aparecem uma vez;
- 2 conferir se a rota retorna ao início;
- 3 somar os custos das arestas;
- 4 verificar se o custo total é  $\leq k$ .

Essa verificação é polinomial.

$$TSP_{\text{decisao}} \in NP$$



## Relação entre P e NP

Todo problema em  $P$  também está em  $NP$ .

Por quê?

Se consigo resolver um problema em tempo polinomial, então também consigo verificar uma solução em tempo polinomial.

$$P \subseteq NP$$

A grande pergunta é:

$$P = NP?$$

Ou seja: todo problema cuja solução pode ser verificada rapidamente também pode ser resolvido rapidamente?

# NP-difícil

## Conceito

Um problema  $B$  é NP-difícil quando todo problema de  $NP$  pode ser reduzido a  $B$  em tempo polinomial.

$$\forall A \in NP, \quad A \leq_p B$$

## Interpretação

$B$  é pelo menos tão difícil quanto qualquer problema de  $NP$ .

## Cuidado

Um problema NP-difícil não precisa pertencer a  $NP$ . Ele pode nem ser um problema de decisão.

# NP-completo

## Conceito

Um problema  $B$  é NP-completo quando satisfaz duas condições:

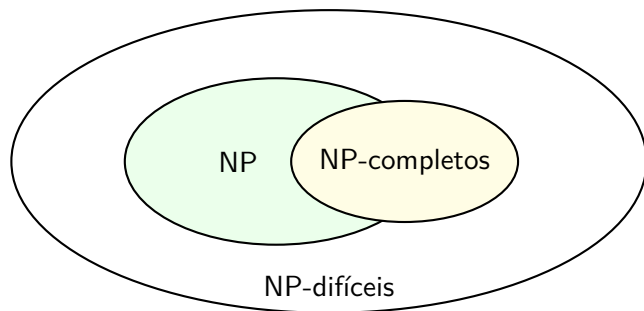
- 1  $B \in NP$ ;
- 2  $B$  é NP-difícil.

$$NP\text{-}completo = NP\text{-}difícil + \text{pertence a } NP$$

## Síntese

Problemas NP-completos são os problemas mais difíceis dentro da classe NP.

## Diferença visual



- NP-completo: está em NP e é NP-difícil.
- NP-difícil: pode estar fora de NP, especialmente se não for problema de decisão.

## Como provar que um problema é NP-difícil?

Na prática, não reduzimos todos os problemas de  $NP$  um por um. Usamos um problema  $A$  que já sabemos ser NP-completo.

$A$  é NP-completo

Se conseguirmos mostrar:

$$A \leq_p B$$

então:

$B$  é NP-difícil

### Síntese

Para provar que  $B$  é difícil, reduzimos um problema difícil conhecido para  $B$ .

## Como provar que um problema é NP-completo?

Para provar que  $B$  é NP-completo:

- ① Mostre que  $B \in NP$ .
  - ▶ Ou seja, uma solução candidata pode ser verificada em tempo polinomial.
- ② Escolha um problema NP-completo conhecido  $A$ .
- ③ Mostre que  $A \leq_p B$ .
  - ▶ Ou seja,  $B$  é pelo menos tão difícil quanto  $A$ .

$$B \in NP \quad + \quad A \leq_p B \Rightarrow B \text{ é NP-completo}$$

## Ciclo Hamiltoniano em NP

Já vimos que *HAM* está em *NP*.

Se alguém fornece uma sequência de vértices, verificamos em tempo polinomial se ela:

- visita todos os vértices;
- não repete vértices;
- usa apenas arestas existentes;
- retorna ao vértice inicial.

Portanto:

$$HAM \in NP$$

## Ciclo Hamiltoniano é NP-difícil

Para mostrar a ideia, podemos usar um problema já conhecido como NP-completo:

$$HAM-PATH_{s,t}$$

Esse problema pergunta:

Existe um caminho Hamiltoniano de  $s$  até  $t$ ?

Se mostramos:

$$HAM-PATH_{s,t} \leq_p HAM-CYCLE$$

então propagamos a dificuldade para Ciclo Hamiltoniano.



## Redução: Hamiltonian Path para Hamiltonian Cycle

Entrada original:

$$(G, s, t)$$

Construção de  $G'$ :

- 1 copie todos os vértices e arestas de  $G$ ;
- 2 adicione um novo vértice  $x$ ;
- 3 conecte  $x$  apenas a  $s$  e a  $t$ .

$$V' = V \cup \{x\}$$

$$E' = E \cup \{(x, s), (x, t)\}$$

## Por que essa redução funciona?

Se existe um caminho Hamiltoniano de  $s$  até  $t$ :

$$s \rightarrow \cdots \rightarrow t$$

então, em  $G'$ , adicionamos:

$$t \rightarrow x \rightarrow s$$

formando um ciclo Hamiltoniano.

No sentido inverso, se  $G'$  possui um ciclo Hamiltoniano, o vértice  $x$  só pode aparecer entre  $s$  e  $t$ , pois ele só é ligado a esses dois vértices.

Removendo  $x$ , sobra um caminho Hamiltoniano de  $s$  até  $t$  em  $G$ .

## Tempo da redução e conclusão

A transformação adiciona:

- um vértice;
- duas arestas.

Logo, é claramente polinomial.

Como:

$$HAM-PATH_{s,t} \leq_p HAM-CYCLE$$

$HAM-CYCLE$  é NP-difícil.

E como:

$$HAM-CYCLE \in NP$$

concluimos:

$HAM-CYCLE$  é NP-completo

# TSP de decisão é NP-completo

Já mostramos:

$$HAM \leq_p TSP_{\text{decisao}}$$

Também sabemos que  $HAM$  é NP-completo. Portanto, todo problema em  $NP$  reduz para  $HAM$ . Pela transitividade, todo problema em  $NP$  reduz para  $TSP_{\text{decisao}}$ .

$$TSP_{\text{decisao}} \text{ é NP-difícil}$$

Além disso, uma rota candidata pode ser verificada em tempo polinomial, então:

$$TSP_{\text{decisao}} \in NP$$

Logo:

$$TSP_{\text{decisao}} \text{ é NP-completo}$$

# TSP de otimização é NP-difícil

Agora considere a versão de otimização:

Qual é a menor rota que visita todas as cidades?

Essa versão não é um problema de decisão, pois sua saída não é SIM ou NÃO. Mesmo assim, ela é NP-difícil.

## Ideia

Se eu tivesse um algoritmo polinomial que encontra a melhor rota, eu poderia resolver a versão de decisão em tempo polinomial.

## Redução: TSP decisão para TSP otimização

Suponha que existe um algoritmo:

$$OPT\_TSP(G)$$

que devolve o menor custo de uma rota em tempo polinomial.

Para resolver  $TSP_{\text{decisao}}$ :

- ① receba  $(G, k)$ ;
- ② calcule  $c = OPT\_TSP(G)$ ;
- ③ se  $c \leq k$ , responda SIM;
- ④ caso contrário, responda NÃO.

Assim:

$$TSP_{\text{decisao}} \leq_p TSP_{\text{otimizacao}}$$

## Conclusão sobre TSP de otimização

Como  $TSP_{\text{decisao}}$  é NP-completo e:

$$TSP_{\text{decisao}} \leq_p TSP_{\text{otimizacao}}$$

então:

|                                                |
|------------------------------------------------|
| $TSP_{\text{otimizacao}} \text{ é NP-difícil}$ |
|------------------------------------------------|

Por que não chamamos de NP-completo?

Porque  $NP$  é uma classe de problemas de decisão. A versão de otimização do TSP não pede uma resposta SIM/NÃO; ela pede a melhor rota.

## Exemplos simples

| Problema           | Classe      | Por quê?                                                |
|--------------------|-------------|---------------------------------------------------------|
| Caminho em grafo   | $P$         | BFS/DFS resolve em tempo polinomial                     |
| Ciclo Hamiltoniano | NP-completo | solução é verificável e é NP-difícil                    |
| TSP de decisão     | NP-completo | está em NP e recebe redução de <i>HAM</i>               |
| TSP de otimização  | NP-difícil  | resolve a versão de decisão, mas não é problema SIM/NÃO |



## O que aconteceria se TSP estivesse em P?

Se encontrássemos um algoritmo polinomial para  $TSP_{\text{decisao}}$ , então:

$$TSP_{\text{decisao}} \in P$$

Como  $TSP_{\text{decisao}}$  é NP-completo, todo problema em  $NP$  reduziria a ele. Logo, todo problema de  $NP$  também teria algoritmo polinomial.

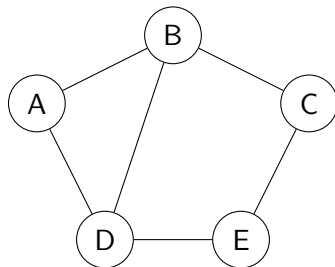
$$\boxed{P = NP}$$

### Síntese

Por isso resolver um único problema NP-completo em tempo polinomial resolveria uma das maiores questões da Computação.

## Atividade guiada: construir a instância TSP

Considere o grafo:



Peça aos alunos:

- 1 listar as arestas originais;
- 2 completar o grafo;
- 3 atribuir peso 1 às arestas originais e peso 2 às demais;
- 4 definir  $k$ ;
- 5 procurar uma rota de custo  $\leq k$ .

## Resolução da atividade guiada

O grafo possui 5 vértices, então:

$$k = 5$$

Uma rota possível é:

$$A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A$$

Todas essas arestas existem no grafo original, então cada uma tem peso 1.

$$1 + 1 + 1 + 1 + 1 = 5$$

Logo, a instância TSP responde SIM.

Portanto, o grafo original possui ciclo Hamiltoniano.

## Síntese: resolver, verificar e reduzir

| Conceito  | Ideia                                                          |
|-----------|----------------------------------------------------------------|
| Resolver  | encontrar a resposta para a instância                          |
| Verificar | conferir uma solução candidata                                 |
| Reduzir   | transformar uma instância de um problema em instância de outro |

### Ponto central

Um problema pode ter solução verificável em tempo polinomial sem conhecermos algoritmo polinomial para resolvê-lo.

## Síntese: classes

- $P$ : problemas de decisão resolvidos em tempo polinomial.
- $NP$ : problemas de decisão cujas soluções podem ser verificadas em tempo polinomial.
- NP-difícil: todo problema de  $NP$  reduz para ele em tempo polinomial.
- NP-completo: está em  $NP$  e é NP-difícil.

$$P \subseteq NP$$

$$NP\text{-completo} = NP\text{-difícil} + \in NP$$

# Síntese: reduções

$$A \leq_p B$$

significa:

- transformo instâncias de  $A$  em instâncias de  $B$ ;
- a transformação é feita em tempo polinomial;
- a resposta é preservada;
- se  $B$  for fácil, então  $A$  também será fácil;
- se  $A$  é difícil, então  $B$  é pelo menos tão difícil quanto  $A$ .

## Mensagem final

Reduções são a ponte entre problemas conhecidos e problemas novos.

$$HAM-PATH \leq_p HAM-CYCLE \leq_p TSP_{\text{decisao}} \leq_p TSP_{\text{otimizacao}}$$

Essa cadeia mostra como a dificuldade de um problema pode ser transferida para outro por meio de transformações polinomiais.